Concurrent Data Structures in Java

HPC Programming Assignment 2

Performance and scalability report

Experiment date: 2026-10-08

This report compares lazy/lock-based and non-blocking implementations using fixed-duration concurrent workloads on the IT Lab systems. All speedups use a one-thread baseline from the same implementation, problem size, and workload.

Results at a glance

Problem	Highest observed speedup	Configuration
Q1: Concurrent doubly linked list	12.97x	lockfree, N=1000, 100C-0I-0D, 16 threads
Q2: Concurrent stack	10.89x	treiber, N=1000, 100C-0I-0D, 16 threads
Q3: Concurrent queue	10.61x	ms, N=1000, 100C-0I-0D, 16 threads
Q4: Elimination-backoff stack	10.99x	elimination, N=1000, 100C-0I-0D, 16 threads

> These are observed maxima across the tested configurations, not claims of general scaling behavior. Full per-run measurements and averaged data are provided separately.

Machines: three identical Intel Core i7-12700 systems, 12 cores / 20 logical CPUs, 15.3 GiB RAM, Ubuntu, OpenJDK 1.8.0_352.

Method and implementation notes

Experimental protocol

- Problem sizes: N = 1,000 and 10,000, with 50% prefilled before each run.
- Workloads: 0C-50I-50D, 50C-25I-25D, and 100C-0I-0D.
- Thread counts: 1, 2, 4, 6, 8, 10, 12, 14, and 16; five repetitions per configuration.
- Each timed interval targets 10 seconds and uses System.nanoTime(). Prefill, deterministic workload traces, thread creation/setup, and CSV output are outside the measured workload interval.
- Throughput is completed operation calls divided by the measured elapsed seconds, in millions of operations per second (MOPS). Speedup is the ratio of average throughput at P threads to the matching one-thread average.

Algorithms and linearization points

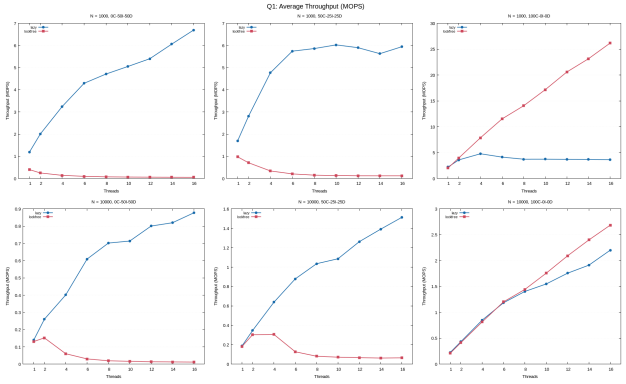
- **Q1 list:** the lazy list inserts at the protected predecessor-link update and deletes at the logical mark; unsuccessful insert/delete and contains linearize at the decision made in the validated, locked window. The non-blocking version publishes immutable doubly linked snapshots with one state CAS; successful updates linearize at that CAS, while unsuccessful updates and contains linearize at the observed snapshot read. It has no transient mark/help/unlink phase: delete omits the key from a new version and atomically publishes that version. In-flight readers may finish on an older immutable version. Copy-on-write updates are O(N), a deliberate correctness-first tradeoff rather than the Sundell-Tsigas linked-node algorithm.
- **Q2 stack:** lock-based push/pop linearize at the protected top mutation, with empty-pop and peek at the protected top observation. Treiber push/pop linearize at a successful top CAS; empty-pop and peek linearize at the atomic top observation.
- **Q3 queue:** lock-based enqueue/dequeue linearize at the protected link/head update, with empty-dequeue and peek at the protected head observation. Michael-Scott enqueue linearizes at the successful tail.next CAS and successful dequeue at the head CAS; empty-dequeue and peek linearize at the validated head/next observation.
- **Q4 elimination stack:** operations that use the central stack linearize at the Treiber top CAS (or the top observation for empty-pop/peek). A matched push/pop pair linearizes at the successful WAITING-to-MATCHED CAS on its shared elimination offer.

Each recorded benchmark run passed worker and post-run structure validation. The separate correctness suite includes sequential empty/populated semantics, duplicate and near-sentinel list keys, exact concurrent list membership, stack value preservation under concurrent pushes, queue value preservation and per-producer FIFO order, paired elimination-stack push/pop value delivery, and mixed-operation stress. These checks are

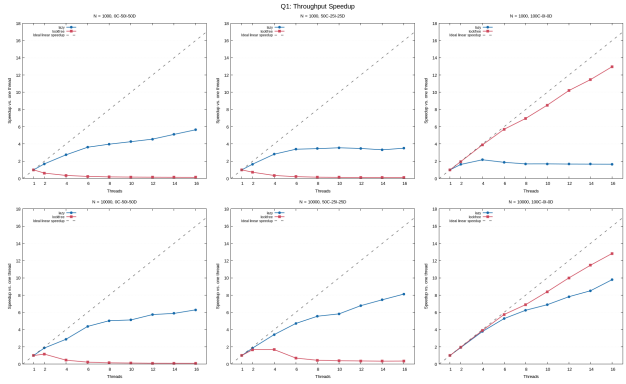
not a formal linearizability-history proof. CAS retry counts are not instrumented; contention and coherence effects are discussed qualitatively.

Q1: Concurrent doubly linked list

Throughput and speedup are shown for both implementations, all three workload mixes, and both problem sizes. The speedup plots include the ideal linear reference.



Average throughput (MOPS).



Throughput speedup versus one thread.

Representative mixed-workload results, N = 10,000

Implementation	Threads	Avg throughput (MOPS)	Speedup
lazy	1	0.1864	1.000x
lazy	4	0.6386	3.427x
lazy	8	1.0353	5.556x
lazy	16	1.5142	8.125x
lockfree	1	0.1813	1.000x
lockfree	4	0.3057	1.686x
lockfree	8	0.0812	0.448x
lockfree	16	0.0648	0.358x

lazy: peak average throughput 6.6920 MOPS at N=1000, 0C-50I-50D, 16 threads.

At 16 threads, **lazy** averaged 5.84x speedup across the six size/workload settings (range 1.65x to 9.80x); 0 of six settings were below its one-thread throughput.

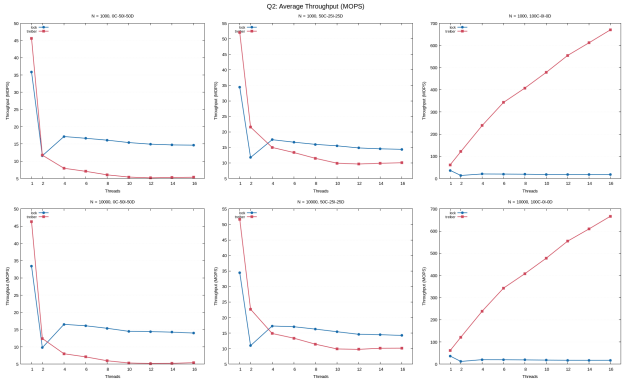
lockfree: peak average throughput 26.1707 MOPS at N=1000, 100C-0I-0D, 16 threads.

At 16 threads, **lockfree** averaged 4.42x speedup across the six size/workload settings (range 0.09x to 12.97x); 4 of six settings were below its one-thread throughput.

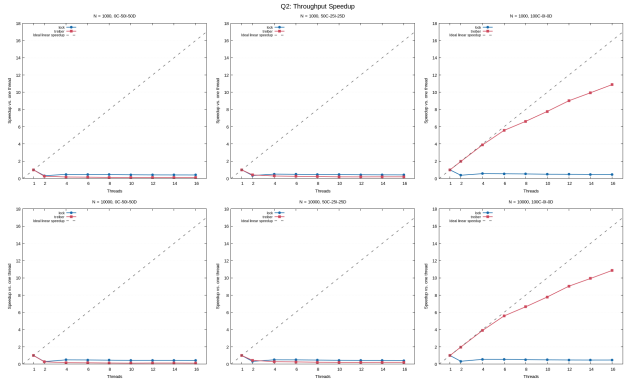
> Interpret scaling in light of the operation mix. A read-only mix avoids update coordination; update-heavy cases expose lock serialization, CAS retries, cache-line ownership traffic, and scheduler effects. An elimination array can reduce central-stack contention only when complementary push/pop operations meet; it may add overhead at lower contention.

Q2: Concurrent stack

Throughput and speedup are shown for both implementations, all three workload mixes, and both problem sizes. The speedup plots include the ideal linear reference.



Average throughput (MOPS).



Throughput speedup versus one thread.

Representative mixed-workload results, N = 10,000

Implementation	Threads	Avg throughput (MOPS)	Speedup
lock	1	34.4541	1.000x
lock	4	17.2911	0.502x
lock	8	16.2834	0.473x
lock	16	14.2436	0.413x
treiber	1	51.6454	1.000x
treiber	4	14.8700	0.288x
treiber	8	11.4170	0.221x
treiber	16	10.1469	0.196x

lock: peak average throughput 36.4349 MOPS at N=10000, 100C-0I-0D, 1 threads.

At 16 threads, **lock** averaged 0.44x speedup across the six size/workload settings (range 0.41x to 0.48x); 6 of six settings were below its one-thread throughput.

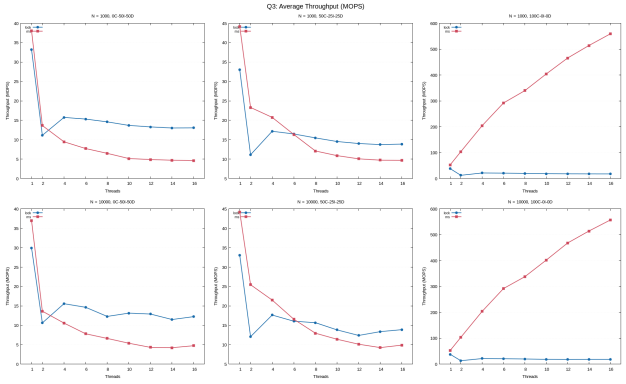
treiber: peak average throughput 670.0176 MOPS at N=1000, 100C-0I-0D, 16 threads.

At 16 threads, **treiber** averaged 3.73x speedup across the six size/workload settings (range 0.12x to 10.89x); 4 of six settings were below its one-thread throughput.

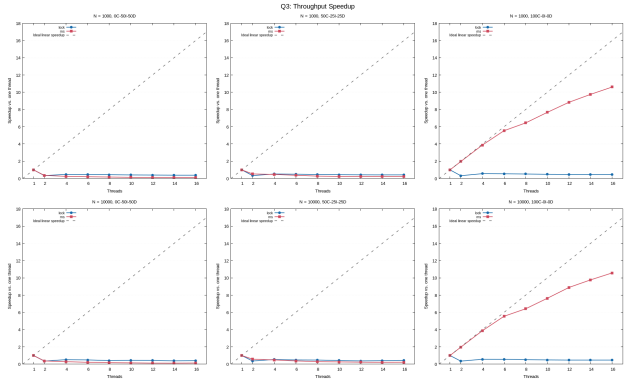
> Interpret scaling in light of the operation mix. A read-only mix avoids update coordination; update-heavy cases expose lock serialization, CAS retries, cache-line ownership traffic, and scheduler effects. An elimination array can reduce central-stack contention only when complementary push/pop operations meet; it may add overhead at lower contention.

Q3: Concurrent queue

Throughput and speedup are shown for both implementations, all three workload mixes, and both problem sizes. The speedup plots include the ideal linear reference.



Average throughput (MOPS).



Throughput speedup versus one thread.

Representative mixed-workload results, N = 10,000

Implementation	Threads	Avg throughput (MOPS)	Speedup
lock	1	33.0329	1.000x
lock	4	17.6473	0.534x
lock	8	15.6786	0.475x
lock	16	13.8852	0.420x
ms	1	44.1774	1.000x
ms	4	21.5074	0.487x
ms	8	12.9704	0.294x
ms	16	9.9005	0.224x

lock: peak average throughput 38.0589 MOPS at N=1000, 100C-0I-0D, 1 threads.

At 16 threads, **lock** averaged 0.43x speedup across the six size/workload settings (range 0.39x to 0.47x); 6 of six settings were below its one-thread throughput.

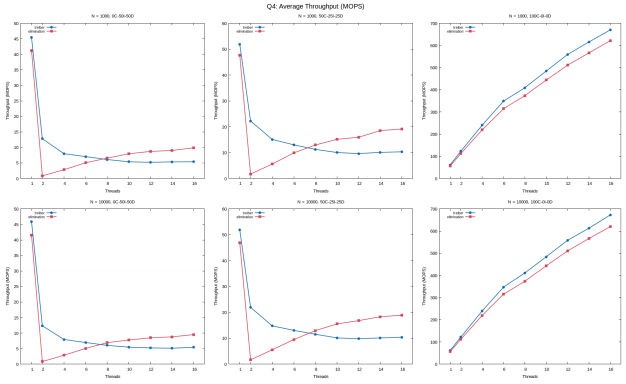
ms: peak average throughput 558.9839 MOPS at N=1000, 100C-0I-0D, 16 threads.

At 16 threads, **ms** averaged 3.65x speedup across the six size/workload settings (range 0.12x to 10.61x); 4 of six settings were below its one-thread throughput.

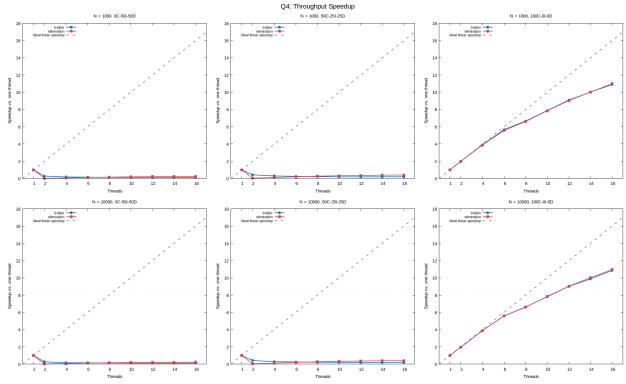
> Interpret scaling in light of the operation mix. A read-only mix avoids update coordination; update-heavy cases expose lock serialization, CAS retries, cache-line ownership traffic, and scheduler effects. An elimination array can reduce central-stack contention only when complementary push/pop operations meet; it may add overhead at lower contention.

Q4: Elimination-backoff stack

Throughput and speedup are shown for both implementations, all three workload mixes, and both problem sizes. The speedup plots include the ideal linear reference.



Average throughput (MOPS).



Throughput speedup versus one thread.

Representative mixed-workload results, N = 10,000

Implementation	Threads	Avg throughput (MOPS)	Speedup
treiber	1	51.9210	1.000x
treiber	4	14.7875	0.285x
treiber	8	11.5372	0.222x
treiber	16	10.3990	0.200x
elimination	1	46.9748	1.000x
elimination	4	5.5206	0.118x
elimination	8	12.9512	0.276x
elimination	16	18.9109	0.403x

treiber: peak average throughput 672.8907 MOPS at N=10000, 100C-0I-0D, 16 threads.

At 16 threads, **treiber** averaged 3.73x speedup across the six size/workload settings (range 0.12x to 10.90x); 4 of six settings were below its one-thread throughput.

elimination: peak average throughput 621.3997 MOPS at N=1000, 100C-0I-0D, 16 threads.

At 16 threads, **elimination** averaged 3.87x speedup across the six size/workload settings (range 0.23x to 10.99x); 4 of six settings were below its one-thread throughput.

At 16 threads, elimination outperformed Treiber in 4 of six matched configurations; the throughput ratio ranged from 0.92x to 1.86x, so its benefit was workload-dependent.

> Interpret scaling in light of the operation mix. A read-only mix avoids update coordination; update-heavy cases expose lock serialization, CAS retries, cache-line ownership traffic, and scheduler effects. An elimination array can reduce central-stack

> contention only when complementary push/pop operations meet; it may add overhead at lower contention.

Interpretation and limitations

Compare each curve within its host and matching implementation, size, and workload. Read-heavy structures may scale differently from update-heavy structures: lock-based paths serialize on shared locks, while non-blocking paths can experience failed CAS retries and cache-coherence traffic on hot references. More threads can improve aggregate throughput, saturate, or regress depending on the critical path and scheduling overhead.

The Q1 copy-on-write doubly linked snapshot is expected to incur high update costs as N grows because a successful update rebuilds the full version. Q4 elimination is workload- and contention-dependent; its results should be judged against Treiber using the update-intensive workloads and higher thread counts, not presumed beneficial.

Per-run raw operation counts, elapsed times, and verification flags are supplied in CSV. Averages and speedups are independently validated from those raw rows. No absolute cross-host algorithm ranking is intended, even though the three hosts reported the same model.

Reproducibility: Java source, correctness test, launch scripts, GNUplot script, machine inventory, raw data, averages, validation output, and plots accompany this report. Compiled class files are excluded from the submission package.